Abstract Syntax Trees and SSA Intermediate Representation

Modern compiler backends transform raw source code through several phases: lexical analysis, syntactic parsing, semantic analysis, intermediate code generation, optimization passes, and target machine code generation. The parser produces an Abstract Syntax Tree (AST) capturing the nested hierarchical grammar of expressions, statements, and blocks.

For aggressive optimization, modern compilers (like LLVM and GCC) convert ASTs into Static Single Assignment (SSA) form. In SSA form, every variable is assigned a value exactly once, and every use is explicitly dominated by its unique definition. Where control flow paths converge (e.g., following an if-else branch or loop header), phi functions mathematically merge competing definitions. SSA vastly simplifies dataflow analyses such as common subexpression elimination, constant folding, dead code elimination, and loop-invariant code motion.